IIS Deployment Guide

ASP.NET Core (Dapper CRUD) Backend + React (Vite) Frontend on IIS

Deployment Steps and Troubleshooting Session Log

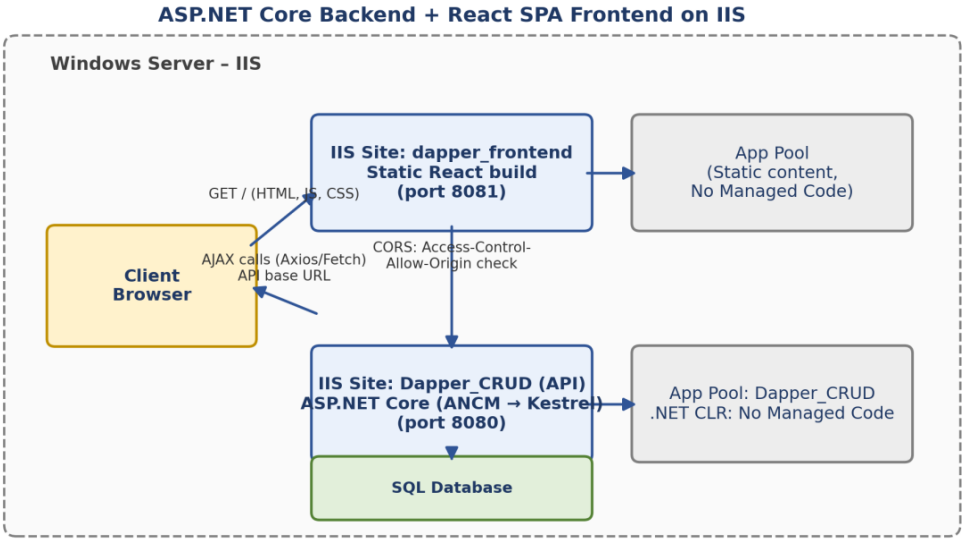

Table of Contents

Part A – Deployment Guide: Backend and Frontend on IIS

Full setup for deploying an ASP.NET Core backend + React SPA frontend on IIS, assuming they will live as two separate sites/apps (typical setup — API on one path/port, static React build served separately or under the same site).

1. Prerequisites on the IIS Server

- Install the IIS role (Web Server (IIS) via Server Manager, or Install-WindowsFeature -Name Web-Server in PowerShell)
- Install the .NET Hosting Bundle (not just the runtime) — this includes the ASP.NET Core Module (ANCM) that lets IIS talk to Kestrel. Download from Microsoft's .NET download page, matching your app's .NET version.
- After installing the hosting bundle, run iisreset to make sure IIS picks up the module.
- Verify: open IIS Manager → check that "ASP.NET Core Module" appears under modules, or check %windir%\System32\inetsrv\config\applicationHost.config for the module entry.

2. Publish the Backend (.NET)

On your dev machine:

```
dotnet publish -c Release -o ./publish
```

This gives you a folder with the compiled DLLs, web.config (auto-generated), and appsettings.json.

3. Deploy Backend to IIS

- Copy the ./publish folder contents to a folder on the server, e.g. C:\inetpub\wwwroot\myapi
- In IIS Manager:
    - Create a new Application Pool (e.g. MyApiPool) → set .NET CLR version = No Managed Code (Kestrel handles the runtime, not IIS's managed pipeline)
    - Create a new Site (or Application under an existing site) pointing to that folder, bind it to a port or hostname
    - Assign it to MyApiPool
- Check web.config in the publish folder has the right processPath/arguments pointing to your app DLL — this is usually auto-generated correctly by dotnet publish.
- Set folder permissions: give the app pool identity (IIS AppPool\MyApiPool) read/execute rights on the folder (and write rights if it logs/writes files).
- Update appsettings.json/appsettings.Production.json with production connection strings, CORS origins (must include your frontend's URL), etc.

4. Build the Frontend (React/Angular/Vue)

```
npm run build
```

This produces a static build (or dist) folder — plain HTML/CSS/JS, no Node needed on the server.

5. Deploy Frontend to IIS

- Copy the build output to e.g. C:\inetpub\wwwroot\myfrontend
- Create another Site (or virtual directory/application) in IIS pointing to that folder
- Static files don't need a .NET app pool with CLR — a plain app pool works fine (can even reuse the default one)
- Important for SPAs: add URL rewriting so client-side routes (e.g. /dashboard) don't 404 on refresh. Install URL Rewrite Module for IIS, then add a web.config in the frontend folder:

```
<configuration>
  <system.webServer>
    <rewrite>
      <rules>
        <rule name="React Routes" stopProcessing="true">
          <match url=".*" />
          <conditions logicalGrouping="MatchAll">
            <add input="{REQUEST_FILENAME}" matchType="IsFile" negate="true" />
            <add input="{REQUEST_FILENAME}" matchType="IsDirectory" negate="true" />
          </conditions>
          <action type="Rewrite" url="/index.html" />
        </rule>
      </rules>
    </rewrite>
  </system.webServer>
</configuration>
```

6. Connect Frontend to Backend

- Make sure your React app's API base URL (env var, e.g. REACT_APP_API_URL) points to the deployed backend's URL/port, and rebuild before deploying if you change it — env vars get baked in at build time for CRA/Vite.
- On the backend, configure CORS in Program.cs/Startup.cs to allow the frontend's origin.

7. Bindings, SSL, Firewall

- Set proper hostname/port bindings for both sites in IIS
- If using HTTPS, bind an SSL certificate to each site
- Open the relevant ports in Windows Firewall if external traffic needs access

8. Test

- Hit the backend directly (e.g. https://yourdomain/api/health) to confirm it's running
- Load the frontend, confirm API calls succeed (check browser console/network tab for CORS or 404 errors)
- Refresh a deep frontend route to confirm the rewrite rule works

If you want, tell me whether frontend and backend are going under the same site (e.g. API under /api subpath) vs separate sites/ports — the IIS structure changes a bit depending on that, and I can give you the exact config either way.

Deployment Architecture

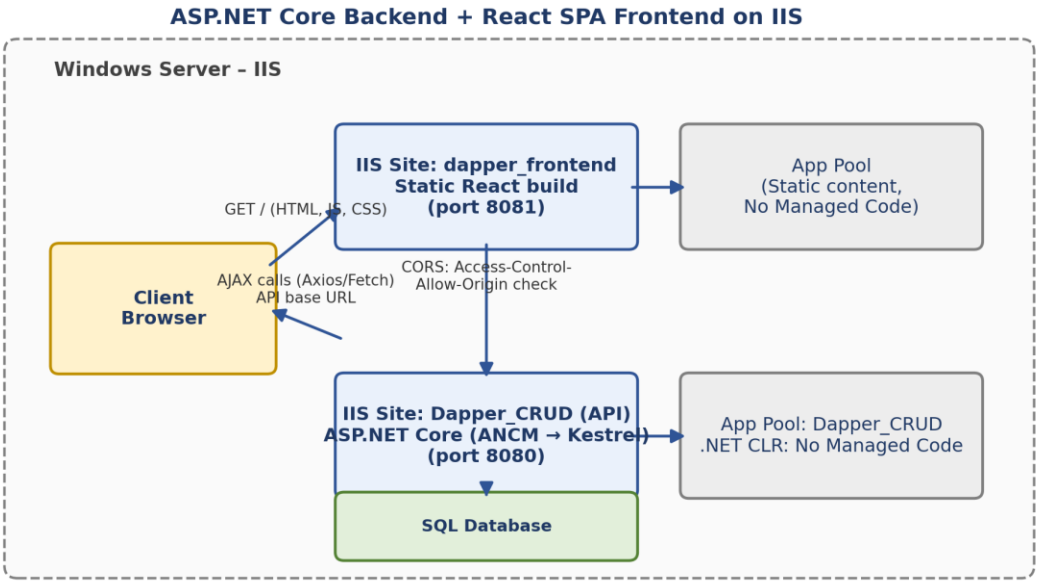

Figure 1. Frontend and backend deployed as separate IIS sites, communicating over HTTP with CORS enabled.

Part B – Deployment Session Log (Chronological Q&A)

The following is the complete record of the deployment and troubleshooting session, preserved exactly as exchanged, organized under numbered topic headings for readability.

B.1 Publishing via Visual Studio "Publish" Instead of the CLI

> **USER**
> *Instead of this :-*
> *dotnet publish -c Release -o ./publish*
> *we can click on publish also?*

Response

Got it — so the backend's already live and working on IIS, you just need to add the frontend now. Here's the part you actually need:

1. Build the Frontend

On your dev machine, in the frontend project folder:

```
npm run build
```

This creates a build (React/CRA) or dist (Vite) folder with static HTML/CSS/JS files. Before building, make sure your API URL env variable points to your already-deployed backend's URL — it gets baked into the build.

2. Copy Build Files to the Server

Copy the entire contents of that build/dist folder to a new folder on the IIS server, e.g.:

```
C:\inetpub\wwwroot\myfrontend
```

3. Create a New Site/App in IIS for the Frontend

- Open IIS Manager
- Right-click Sites → Add Website (or Add Application if you want it under the same site as the backend, like /app)
- Set:
    - Site name: e.g. MyFrontend
    - Physical path: C:\inetpub\wwwroot\myfrontend
    - Binding: pick a port (e.g. 8081) or hostname not already used by the backend site
- Application pool: static files don't need the .NET CLR, so this can even use a plain/default app pool (not the one your backend uses)

4. Add the SPA Rewrite Rule (Important)

Without this, refreshing any route other than / will 404. Install the URL Rewrite module for IIS if not already there, then create a web.config file directly inside your myfrontend folder:

```
<configuration>
  <system.webServer>
    <rewrite>
      <rules>
        <rule name="React Routes" stopProcessing="true">
          <match url=".*" />
          <conditions logicalGrouping="MatchAll">
            <add input="{REQUEST_FILENAME}" matchType="IsFile" negate="true" />
            <add input="{REQUEST_FILENAME}" matchType="IsDirectory" negate="true" />
          </conditions>
          <action type="Rewrite" url="/index.html" />
        </rule>
      </rules>
    </rewrite>
  </system.webServer>
</configuration>
```

5. Check CORS on the Backend

Since frontend and backend are separate sites/ports, the backend needs to allow the frontend's origin in its CORS policy (Program.cs/Startup.cs). If you skip this, the frontend will load but API calls will fail in the browser console with a CORS error.

6. Test

- Visit the frontend site's URL/port in a browser
- Open dev tools → Network tab → confirm API calls to the backend succeed
- Try refreshing on a non-root route to confirm the rewrite rule works

One thing worth confirming: do you want the frontend on its own port/site (like :8081), or should it be reachable on the same domain as the backend (e.g. backend under /api, frontend at root)? That changes how you set the binding and whether you need the rewrite rule to also proxy /api calls through to the backend site.

B.2 Confirming the Frontend-Only Deployment Scope

> **USER**
> *Actually I had aldready published that it was visible like the json data now i want the whole frontend part also*

Response

Got it — so the backend's already live and working on IIS, you just need to add the frontend now. Here's the part you actually need:

(Guidance repeated: Build the frontend with npm run build → copy build/dist output to C:\inetpub\wwwroot\myfrontend → create the IIS site/app → add the SPA rewrite rule → verify CORS on the backend → test the deployed frontend, exactly as set out in section B.1 above.)

One thing worth confirming: do you want the frontend on its own port/site (like :8081), or should it be reachable on the same domain as the backend (e.g. backend under /api, frontend at root)? That changes how you set the binding and whether you need the rewrite rule to also proxy /api calls through to the backend site.

B.3 Locating the Build Output Folder

> **USER**
> *I have done with npm run build now where i can find the build ?*

Response

It'll be created right inside your frontend project's root folder — same level as package.json, src, node_modules, etc.

- Create React App (CRA): folder named build
- Vite: folder named dist

So if your project is at, say, C:\Projects\my-frontend, look for:

```
C:\Projects\my-frontend\build     (CRA)
C:\Projects\my-frontend\dist      (Vite)
```

If you're not sure which tool you're using, check package.json — the scripts section will show something like “build”: “react-scripts build” (CRA) or “build”: “vite build” (Vite), which tells you which output folder to expect.

Once you find it, that entire folder's contents (not the folder itself) is what gets copied to C:\inetpub\wwwroot\myfrontend on the server.

B.4 First Build Attempt — Missing Output Folder

> **USER**
> *BUILD :- npm run build*

Response

Looking at your explorer — there's no dist folder in your project at all. That means npm run build either didn't actually run, or it failed before creating output.

Here's how to fix this:

1. Open a Terminal in This Exact Project Folder

In VS Code: Terminal → New Terminal (it'll open at dapper_react since that's your project root — confirm the path shown matches where package.json is).

2. Run the Build Command Again and Watch the Output

```
npm run build
```

Don't let it run in the background — watch what prints in the terminal. You're looking for either:

- ✅Success: something like "✓built in 1.2s" followed by a list of output files
- ❌Error: red text with a stack trace / error message

3. If It Errors Out

Copy the exact error text here and I'll help you fix it. Common culprits: a broken import in your code, a missing dependency, or an ESLint/type error blocking the build.

4. If It Says Success but You Still Don't See dist

- Refresh the Explorer panel in VS Code (small refresh icon at top of Explorer, visible in your screenshot next to the folder-add icons)
- Check you're not looking in a different folder than where you ran the command — run pwd (or cd with no args on Windows cmd) right before npm run build to confirm your terminal's current directory matches dapper_react

Go ahead and run it now and paste what the terminal shows.

B.5 Successful Build Output

> **USER**
> *Now created :-*
>
> *D:\Dapper\Dapper-API-React\Dapper_React\dapper_react>npm run build*
> *> dapper_react@0.0.0 build*
> *> vite build*
> *vite v8.1.3 building client environment for production...*
> *✓85 modules transformed.*
> *computing gzip size...*
> *dist/index.html 0.46 kB | gzip: 0.30 kB*
> *dist/assets/index-D2Y52sNM.css 231.76 kB | gzip: 31.16 kB*
> *dist/assets/index-QPz9VM9c.js 282.42 kB | gzip: 92.39 kB*
> *✓built in 758ms*
>
> *now tell what to do?*

Response

Build succeeded. Your dist folder is now here:

```
D:\Dapper\Dapper-API-React\Dapper_React\dapper_react\dist
```

It contains index.html and an assets folder with your JS/CSS — that's your complete production build.

Next steps: deploy to IIS

1. Copy the contents of dist (not the folder itself, everything inside it) to a new folder on your IIS server, e.g.:

```
C:\inetpub\wwwroot\dapper_frontend
```

So you should end up with C:\inetpub\wwwroot\dapper_frontend\index.html, C:\inetpub\wwwroot\dapper_frontend\assets\..., etc.

2. Create the site in IIS Manager

- Right-click Sites → Add Website
- Site name: e.g. DapperFrontend
- Physical path: C:\inetpub\wwwroot\dapper_frontend
- Binding: pick a port not already used by your backend API site (e.g. 8081)
- Application pool: any basic pool without .NET CLR requirements is fine (static files only)

3. Add the SPA Rewrite Rule

Since this is a React app with client-side routing, add a web.config file directly inside C:\inetpub\wwwroot\dapper_frontend:

```
<configuration>
  <system.webServer>
    <rewrite>
      <rules>
        <rule name="React Routes" stopProcessing="true">
          <match url=".*" />
          <conditions logicalGrouping="MatchAll">
            <add input="{REQUEST_FILENAME}" matchType="IsFile" negate="true" />
            <add input="{REQUEST_FILENAME}" matchType="IsDirectory" negate="true" />
          </conditions>
          <action type="Rewrite" url="/index.html" />
        </rule>
      </rules>
    </rewrite>
  </system.webServer>
</configuration>
```

(Needs the URL Rewrite module installed on IIS if you haven't already.)

4. Check Your Backend's CORS Settings

Since the frontend will run on a different port than your API, make sure the backend's CORS policy allows this new frontend origin (e.g. http://yourserver:8081).

5. Test

- Browse to the site in IIS Manager, or hit http://yourserver:8081 directly
- Open browser dev tools → Network tab, confirm API calls to your backend succeed without CORS errors
- Refresh a non-root route to confirm the rewrite rule works

Let me know once you've copied the files and created the site in IIS — happy to help with any errors that come up.

B.6 Opening IIS Manager

> **USER**
> *How to open IIS?*

Response

On Windows:

Quickest Way

Press Win + R, type inetmgr, hit Enter.

Alternative Ways

- Start menu: search for “Internet Information Services (IIS) Manager” and click it
- Control Panel: Control Panel → Administrative Tools → Internet Information Services (IIS) Manager

If It's Not Installed / Doesn't Open

That means the IIS feature isn't enabled yet. To install it:

1. Open Control Panel → Programs → Turn Windows features on or off (or search “Turn Windows features on or off” in Start)
2. Check the box for Internet Information Services
3. Expand it and make sure at least these are checked:
    a. Web Management Tools → IIS Management Console
    b. World Wide Web Services → Application Development Features → check ASP.NET Core Hosting related items if listed (or install the .NET Hosting Bundle separately afterward, as mentioned earlier)
4. Click OK and let it install (may need a restart)

Once installed, inetmgr will open the IIS Manager console where you can right-click Sites to add your frontend site.

Are you doing this on your own machine, or is this a separate IIS server you're RDP-ing into? That matters since if it's a remote server, someone may have already set it up (since your backend JSON API is already running on IIS somewhere).

B.7 Clarifying What "Copy" Means

> **USER**
> *Copy means actually what to do ?*

Response

It literally just means copy-pasting the files using File Explorer (or a copy command) — like copying any files from one folder to another on Windows.

Since it sounds like your IIS is on the same machine you're working on (based on your earlier questions), here's exactly what to do:

Using File Explorer (Easiest)

1. Open File Explorer and navigate to:

```
D:\Dapper\Dapper-API-React\Dapper_React\dapper_react\dist
```

1. Go inside the dist folder (so you can see index.html and the assets folder directly)
2. Select everything inside it: press Ctrl + A
3. Copy: Ctrl + C
4. Now navigate to (or create) the destination folder:

```
C:\inetpub\wwwroot\
```

1. Create a new folder here named dapper_frontend (right-click → New → Folder)
2. Open that new dapper_frontend folder
3. Paste: Ctrl + V

End result — when you open C:\inetpub\wwwroot\dapper_frontend, you should directly see:

```
C:\inetpub\wwwroot\dapper_frontend\index.html
C:\inetpub\wwwroot\dapper_frontend\assets\
```

(NOT C:\inetpub\wwwroot\dapper_frontend\dist\index.html — that would be wrong, it means you copied the folder itself instead of its contents)

Alternative: Command Line (If You Prefer)

Open Command Prompt or PowerShell and run:

```
xcopy "D:\Dapper\Dapper-API-React\Dapper_React\dapper_react\dist\*"
"C:\inetpub\wwwroot\dapper_frontend\" /E /I
```

This copies everything from inside dist into dapper_frontend, creating the folder if it doesn't exist.

Try it and let me know once it's done — next step is creating the site in IIS Manager pointing to that folder.

B.8 Verifying the Copied Folder Structure

> **USER**
> *It's like this?*
> *End result — when you open C:\inetpub\wwwroot\dapper_frontend, you should directly see:*
>
> *C:\inetpub\wwwroot\dapper_frontend\index.html*
> *C:\inetpub\wwwroot\dapper_frontend\assets*

Response

Yes, exactly right. That's precisely what we wanted:

- assets folder ✓
- index.html (shown as "index" since your Explorer is hiding file extensions) ✓

This is the correct structure — you copied the contents of dist, not the dist folder itself.

Next Step: Create the Site in IIS Manager

1. Open IIS Manager (inetmgr)
2. In the left panel, right-click Sites → Add Website
3. Fill in:
    a. Site name: dapper_frontend (or any name you like)
    b. Physical path: click the ... button and browse to/select C:\inetpub\wwwroot\dapper_frontend
    c. Binding:
    d. Type: http
    e. Port: pick one not already in use — check what port your backend API is running on first, then pick something different (e.g. if backend is on 5000, use 8081 for frontend)
4. Click OK

Once created, IIS should auto-start the site. Then:

1. In the left panel, click on your new site → in the right "Actions" panel click Browse *:8081 (http) (or whatever port you chose) — this opens it in your browser to test

Let me know what port your backend is currently running on (or send a screenshot of your Sites list in IIS Manager) so we pick a frontend port that won't conflict.

B.9 IP Address and Host Name Fields

> **USER**
> *What to write in IP Address and Host name?*

Response

For a straightforward local deployment, here's what to put:

Field	Recommended value
IP Address	Leave it as All Unassigned (this is the default). This means the site will respond on that port across any IP address the server has — simplest option and works fine unless you specifically need to bind to one network interface.
Host name	Leave it blank — unless you have a specific domain name pointing to this server that you want to use (like frontend.yourcompany.com).

If you leave both IP Address = All Unassigned and Host name = blank, you'll access the site simply via:

```
http://localhost:8081
or
http://<server-IP>:8081
```

from other machines on the network.

When You WOULD Fill in Host Name

Only if:

- You're hosting multiple sites on the same port (like port 80) and need IIS to tell them apart by domain name, or
- You already have a DNS entry / hosts file mapping set up for something like dapper.local or dapper-frontend.company.com pointing to this server

Since this sounds like your first deployment and you're likely just testing/accessing it directly, leave both as default (All Unassigned + blank) and just make sure your Port is unique (not clashing with your backend API site or anything else already running).

Go ahead and click OK — then try browsing to http://localhost:<your-port> to confirm it loads.

B.10 Post-Site-Creation Verification and the SPA Rewrite Rule

> **USER**

> *IP Address*
> *Leave it as All Unassigned (this is the default). This means the site will respond on that port across any IP address the server has — simplest option and works fine unless you specifically need to bind to one network interface.*
>
> *Host name*
> *Leave it blank — unless you have a specific domain name pointing to this server that you want to use (like frontend.yourcompany.com).*
>
> *Leaved it unassigned and click Ok Now what to do after this?*

Response

Good. Now let's verify it and get it fully working.

1. Check the Site Started

In IIS Manager, click on Sites in the left panel — you should see your new site (e.g. dapper_frontend) listed with a green arrow/running status. If it shows stopped, right-click it → Start.

2. Browse to It and Test

Right-click your site → Browse *:8081 (http) (or whatever port you picked). This opens it in your default browser.

What you should see: Your React app's homepage should load.

If you see a 403 error: Default Document isn't finding index.html — check that index.html is directly inside the site's physical path (it is, based on your screenshot).

If you see a 500 error or blank page: Could be a JS/routing issue — open browser dev tools (F12) → Console tab and tell me what error shows up.

3. Add the SPA Rewrite Rule (Important — Do This Even If Step 2 Worked)

Without this, your homepage will load fine, but refreshing on any inner route (like /dashboard or /login) will 404.

Check if URL Rewrite module is installed: in IIS Manager, click your site → look for a “URL Rewrite” icon in the middle pane. If it's not there, download and install it from Microsoft's IIS site (search “IIS URL Rewrite Module download”).

Once installed, create a file named web.config inside C:\inetpub\wwwroot\dapper_frontend (same folder as index.html) with this content:

```
<configuration>
  <system.webServer>
    <rewrite>
```

```
      <rules>
        <rule name="React Routes" stopProcessing="true">
          <match url=".*" />
          <conditions logicalGrouping="MatchAll">
            <add input="{REQUEST_FILENAME}" matchType="IsFile" negate="true" />
            <add input="{REQUEST_FILENAME}" matchType="IsDirectory" negate="true" />
          </conditions>
          <action type="Rewrite" url="/index.html" />
        </rule>
      </rules>
    </rewrite>
  </system.webServer>
</configuration>
```

4. Test API Connectivity

Once the app loads, open dev tools (F12) → Network tab, and check if calls to your backend API succeed. If you see CORS errors in the console, that means the backend needs to allow this frontend's origin (http://localhost:8081 or whatever URL you're using) in its CORS policy.

Try step 2 first and tell me what you see — success, blank page, or an error.

B.11 Backend App Pool Not Started

> **USER**
> *I have to run backend also as you can see frontend is started but backend has not been started*

Response

Right, your backend's app pool (Dapper_CRUD) is stopped — that's why the API isn't responding. Let's fix that.

Start the Backend App Pool

1. In this same Application Pools list, right-click Dapper_CRUD
2. Click Start
3. Confirm the Status column now shows Started

Also Check the Site Itself Is Started (Not Just the App Pool)

An app pool can be running but the site bound to it could still be stopped separately.

1. In the left panel of IIS Manager, click Sites
2. Find your backend site (likely also named something like Dapper_CRUD or similar)
3. If its status isn't Started, right-click it → Start

Why It Stopped in the First Place

Common reasons an app pool stops on its own:

- Rapid-fail protection — if the app crashed multiple times in a short window, IIS auto-stops the pool to prevent a crash loop. Check Event Viewer → Windows Logs → Application for errors around the time it stopped.
- It was manually stopped earlier and just never restarted
- The app pool identity lacks permission to read its files, causing repeated failures

After Starting It, Verify

Hit your backend's API URL directly in the browser (e.g. http://localhost:<backend-port>/api/health or whatever endpoint you used to test the JSON before). Confirm it responds like it used to.

Once that works, go back to your frontend site and confirm the API calls succeed now (F12 → Network tab, look for successful 200 responses instead of failed/red requests).

Let me know what you see after starting it.

B.12 404 on Port 8080 (Site Not Started)

> **USER**
> *After clicking on Browse website on 8080*
>
> *This localhost page can't be found*
> *No webpage was found for the web address: http://localhost:8080/*
> *HTTP ERROR 404*

Response

That 404 is coming from the browser/network level (not IIS's own 404 page), which usually means nothing is actually listening on port 8080 — likely the site itself (not just the app pool) is stopped, or there's a port mismatch.

Let's check a few things:

1. Confirm Which Site Is Bound to Port 8080

In IIS Manager → click Sites in the left panel, and look at the list. Check:

- Which site shows 8080 in its Bindings column
- Is that the frontend or backend?
- What's its Status — Started or Stopped?

Send a screenshot of this Sites list if easy — it'll clear up a lot.

2. If the Site Shows "Started" but Still 404s

Right-click that site → Edit Bindings → confirm the port listed there actually matches 8080 (typos happen, e.g. 8080 vs 8090).

3. Check for a Port Conflict

If two sites (or the backend app) are both trying to use port 8080, only one can actually bind to it — the other silently fails to listen even if IIS shows it as “Started.”

Run this in Command Prompt to see what's actually listening on 8080:

```
netstat -ano | findstr :8080
```

This shows the process ID (PID) using that port. Then check Task Manager → Details tab → find that PID → see what process it belongs to (should be w3wp.exe for IIS if it's working correctly).

4. Double Check You're Browsing the Right Site

If you clicked “Browse” from the frontend site's Actions panel, but frontend's binding is actually a different port (not 8080), IIS Manager's browse button uses whatever port is configured — so worth reconfirming what port your frontend site is actually bound to (Edit Bindings on that site).

Can you share a screenshot of your Sites list (not Application Pools) showing site names, status, and bindings? That'll make this quick to diagnose.

B.13 Confirming Frontend Works, Backend Site Stopped

> **USER**
> *It's backend not working frontend is working*

Response

Got it — good, so frontend is confirmed working and this 404 was just the backend site not started yet on 8080.

Same fix as before, just for the site level this time:

Start the Backend Site

1. In IIS Manager, click Sites in the left panel
2. Find the backend site (bound to port 8080)
3. Right-click it → Start
4. Confirm its Status shows Started

Then also double check its app pool (Dapper_CRUD) is still showing Started in Application Pools — since it was stopped earlier, make sure it didn't flip back to stopped.

Then Test Again

Click Browse *:8080 (http) on that site (or just refresh http://localhost:8080 in your browser). You should get your JSON response back like before, instead of the 404.

Once Backend Is Confirmed Working

Go back to your frontend in the browser, open dev tools (F12) → Network tab, and check if the API calls from the frontend are now succeeding (status 200) or if you're hitting a CORS error — since frontend (8081-ish) and backend (8080) are different ports, the backend needs to explicitly allow that origin.

Let me know what happens after starting the site — and if you hit CORS errors, share the console message and I'll help you fix the CORS config in your Program.cs/Startup.cs.

B.14 404 at Backend Root URL

> **USER**
> *Request URL*
> *http://localhost:8080/*
> *Request Method*
> *GET*
> *Status Code*
> *404 Not Found*
> *Remote Address*
> *[::1]:8080*
> *Referrer Policy*
> *strict-origin-when-cross-origin*

Response

That's actually expected behavior for most Web APIs — a 404 at the root / doesn't mean the backend is broken. ASP.NET Core Web APIs typically don't have anything mapped to the bare root route; they only respond on specific endpoints/controllers.

The site is now running (this is a real IIS 404 response, not “site not found” — meaning IIS successfully routed the request to your app, and your app just doesn't have a / route defined).

Test the Actual Endpoint Instead

What was the specific URL you were hitting before to get your JSON data? Something like:

```
http://localhost:8080/api/something
or
http://localhost:8080/swagger
```

(if Swagger/OpenAPI UI is enabled, that's a good way to see all available routes)

Try that specific endpoint instead of just http://localhost:8080/ and see if you get your JSON back like before.

If you don't remember the exact route, check your Controllers folder in the backend project for [Route(“api/...”)] attributes, or check your frontend's API service file (likely in src/services/) — it'll have the base URL + endpoint paths your React app is calling.

B.15 Employee Controller Source and Correct API Route

USER

```
public class EmployeeController : ControllerBase
{

    IEmpService service;

    public EmployeeController(IEmpService service)
    {
        this.service = service;
    }

    [HttpPost]
    public async Task<IActionResult> AddNewEmp(Employee e)
    {
        await service.AddEmp(e);
        return Ok(new {message= "Emp added Successfully"});
    }


    [HttpGet]
    public async Task<IActionResult> GetEmployees()
    {
        var data = await service.GetEmployees();
        return Ok(data);
    }


    [HttpGet("{id}")]
    public async Task<IActionResult> GetEmployeeById(int id)
    {
        var employee = await service.GetEmployeeById(id);

        if(employee == null)
        {
            return NotFound("Employee Not Found");
        }

        return Ok(employee);
    }
```

```
    [HttpDelete("{id}")]

    public async Task<IActionResult> DeleteEmployee(int id)
    {
        var result = await service.DeleteEmployee(id);

        if(result == 0)
        {
            return NotFound("Employee not found");
        }

        return Ok(new { message = "Employee deleted successfully" });
    }



    [HttpPut("{id}")]
    public async Task<IActionResult> UpdateEmployee(int id, Employee e)
    {
        e.EmployeeId = id;

        var result = await service.UpdateEmployee(e);

        if(result == 0)
        {
            return NotFound("Employee not found");
        }

        return Ok(new { message = "Employee Updated Successfully" });
    }
}
```

Response

This confirms it — there's no route mapped to /, so that 404 is normal. Your actual endpoint is based on the [Route] attribute on the class (likely above what you pasted, probably something like [Route(“api/[controller]”)]).

Given your controller is named EmployeeController, try:

```
http://localhost:8080/api/Employee
```

(GET request — this should hit your GetEmployees() method and return the employee list as JSON)

If that still 404s, open your controller file and check the line just above public class EmployeeController — there should be a [Route(...)] attribute (and likely [ApiController] too). Paste that line if you're unsure, and I'll give you the exact URL to test.

Also worth checking: your appsettings.json on the server — does the connection string in it point to the correct production database (not your local dev DB)? Since this is now running on the server, a wrong connection string is a common reason for a working build to still fail on real requests.

B.16 Backend Confirmed Working but Frontend Not Receiving Data

> **USER**
> *FRONTEND URL IS WORKING BUT BACKEND IS NOT WORKING*

Response

Good progress — backend confirmed working with real data. But look closely at the Network tab in your screenshot: the Employee request row has no Status Code shown and 0 B transferred, with 2.33s duration — that's very different from a normal successful call. This strongly suggests the request either failed (CORS block) or is hanging.

1. Click on That "Employee" Request Row Itself

Click directly on the Employee row in the Network tab → check the Headers tab (Status Code) and the Console tab for any red error text. A CORS failure shows up as a red console error like:

```
Access to XMLHttpRequest at 'http://localhost:8080/api/Employee' from origin
'http://localhost:8081' has been blocked by CORS policy
```

2. Most Likely Cause: CORS

Your frontend (localhost:8081 or whatever port) and backend (localhost:8080) are different origins. If the backend's CORS policy doesn't explicitly allow the frontend's origin, the browser blocks the response even though the API itself works fine (which is exactly why hitting the URL directly in the browser worked, but the app's fetch call doesn't — direct browser navigation isn't subject to CORS, but JS fetch/axios calls are).

Fix: In your backend's Program.cs, check/add CORS config like this:

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowFrontend", policy =>
    {
        policy.WithOrigins("http://localhost:8081") // your actual frontend URL/port
              .AllowAnyHeader()
              .AllowAnyMethod();
    });
```

```
});
```

And make sure it's applied in the middleware pipeline (before app.MapControllers() or similar):

```
app.UseCors("AllowFrontend");
```

After changing this, you'll need to rebuild and republish the backend, then restart the site/app pool in IIS.

3. Also Double-Check Your Frontend's API Base URL

Open your frontend's service file (likely src/services/...) and confirm the base URL matches your backend's actual address exactly — http://localhost:8080/api — no typos, no trailing slash mismatches.

Click on that Employee request in Network tab and tell me what the Console tab shows (any red error), or what Status Code appears in Headers — that'll confirm whether it's CORS or something else.

B.17 Discovering the Stale API Base URL

> **USER**
> *Open your frontend's service file (likely src/services/...) and confirm the base URL matches your backend's actual address exactly — http://localhost:8080/api — no typos, no trailing slash mismatches.*

Response

Found it. Your frontend is calling:

```
https://localhost:7266/api/Employee
```

That's your old local dev backend (Visual Studio's HTTPS launch profile port), not your deployed IIS backend. Since 7266 isn't running anymore on the server, that request just hangs/fails — which matches the 2.33s stuck request with no response you saw earlier.

Fix: Update the API URL

In your src/services/... file, change:

```
const API_URL = "https://localhost:7266/api/Employee";
```

to:

```
const API_URL = "http://localhost:8080/api/Employee";
```

(matching your actual deployed backend site/port in IIS — confirm 8080 is correct)

Then Rebuild and Redeploy the Frontend

Since this URL gets baked into the build at build time, you need to:

1. Save the file
2. Run npm run build again
3. Delete the old contents of C:\inetpub\wwwroot\dapper_frontend (or at least the assets folder + index.html)
4. Copy the new dist contents into that folder again
5. Refresh your browser (hard refresh: Ctrl+Shift+R to bypass cache)

Also Confirm Your CORS Policy Is Applied

You showed the CORS policy definition, but make sure it's also applied in the middleware pipeline in Program.cs — just defining the policy isn't enough:

```
app.UseCors("ReactPolicy");
```

This line needs to be present, and positioned after app.UseRouting() (if present) and before app.MapControllers() / app.UseAuthorization().

Also double check your frontend really is running on port 8081 — if IIS Manager shows a different port for your frontend site's binding, update the CORS WithOrigins(...) to match that exact port instead.

Once you've updated the URL, rebuilt, and redeployed, try loading the frontend again and check the Network tab — you should now see a 200 status with actual data on that Employee request.

B.18 Re-Checking the Request After the URL Fix

Response

Progress — headers should now match if CORS is applied. Let's check what's actually happening now with the request.

1. Open Dev Tools (F12) → Network Tab → Hard Refresh (Ctrl+Shift+R)

Look for the Employee request again and click on it. Check two things:

- A. Status code (in Headers tab) — is it 200, or something else (CORS block, 404, etc.)?
- B. Console tab — any red error text? Specifically look for:
    - A CORS error (blocked by CORS policy)
    - A mixed content error (this is likely, explained below)

2. Likely Culprit: HTTP vs HTTPS Mismatch

Your frontend URL earlier was mentioned as http://localhost:8081 (plain HTTP), and now you're calling http://localhost:8080 (also HTTP) — so that should be fine, no mixed-content issue there. But double check your current browser URL bar — if you're viewing the frontend via https:// instead of http://, that could cause silent blocking on the API call. Look at the address bar in your browser right now and confirm it starts with http:// not https://.

3. Confirm app.UseCors(...) Is Actually Called

You showed the CORS policy definition again, but I still need to confirm the usage line exists in Program.cs. Search your Program.cs for UseCors — do you see this line present?

```
app.UseCors("ReactPolicy");
```

If it's missing, the policy is defined but never applied, and requests will still fail. Paste your full Program.cs (or at least the middleware section, from var app = builder.Build(); down to app.Run();) so I can confirm the order is correct.

4. Confirm You Rebuilt AND Redeployed

Just to double check — after saving the API_URL change, did you:

```
npm run build
```

again, then copy the new dist contents over the old ones in C:\inetpub\wwwroot\dapper_frontend? If the old build is still sitting there, the browser could still be running the old JS bundle with the 7266 URL baked in (check this in Network tab too — click the failing request and see what URL it's actually hitting under Headers → Request URL).

Paste your Program.cs middleware section and the Console/Network error details — that'll pinpoint it fast.

B.19 CORS Error Confirmed in Console

> **USER**
> *From console tab:-*
>
> *Access to XMLHttpRequest at 'http://localhost:8080/api/Employee' from origin 'http://localhost:8081' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.*
> *index-CQ-o6mNN.js:19 Error loading employees: AxiosError: Network Error*
> *at h.onerror (index-CQ-o6mNN.js:13:17186)*
> *i @ index-CQ-o6mNN.js:19*
> *await in i*

> *(anonymous) @ index-CQ-o6mNN.js:19*
> *Hc @ index-CQ-o6mNN.js:8*
> *Dl @ index-CQ-o6mNN.js:8*
> *El @ index-CQ-o6mNN.js:8*
> *Dl @ index-CQ-o6mNN.js:8*
> *El @ index-CQ-o6mNN.js:8*
> *Dl @ index-CQ-o6mNN.js:8*
> *El @ index-CQ-o6mNN.js:8*
> *Dl @ index-CQ-o6mNN.js:8*
> *El @ index-CQ-o6mNN.js:8*
> *Dl @ index-CQ-o6mNN.js:8*
> *El @ index-CQ-o6mNN.js:8*
> *Dl @ index-CQ-o6mNN.js:8*
> *El @ index-CQ-o6mNN.js:8*
> *Dl @ index-CQ-o6mNN.js:8*
> *El @ index-CQ-o6mNN.js:8*
> *Dl @ index-CQ-o6mNN.js:8*
> *El @ index-CQ-o6mNN.js:8*
> *Dl @ index-CQ-o6mNN.js:8*
> *El @ index-CQ-o6mNN.js:8*
> *Dl @ index-CQ-o6mNN.js:8*
> *Uu @ index-CQ-o6mNN.js:8*
> *(anonymous) @ index-CQ-o6mNN.js:8*
> *re @ index-CQ-o6mNN.js:1*
> *postMessage*
> *ie @ index-CQ-o6mNN.js:1*
> *re @ index-CQ-o6mNN.js:1*
> *postMessage*
> *ie @ index-CQ-o6mNN.js:1*
> *e.unstable_scheduleCallback @ index-CQ-o6mNN.js:1*
> *sd @ index-CQ-o6mNN.js:8*
> *od @ index-CQ-o6mNN.js:8*
> *(anonymous) @ index-CQ-o6mNN.js:8*
> *index-CQ-o6mNN.js:13 GET http://localhost:8080/api/Employee net::ERR_FAILED 200 (OK)*

Response

This confirms it clearly: the request actually succeeds at the server level (200 OK), but the browser blocks the response because no CORS headers are being sent back. That means the ReactPolicy you defined isn't actually being applied by the middleware — either app.UseCors(...) is missing, in the wrong position, or the backend wasn't rebuilt/republished after adding it.

Fix: Verify and Fix Program.cs

Open your backend's Program.cs and check the full middleware pipeline. It needs to look like this (order matters a lot in ASP.NET Core):

```
var builder = WebApplication.CreateBuilder(args);

// ... your existing services (controllers, DbContext, DI, etc.)

builder.Services.AddCors(options =>
{
    options.AddPolicy("ReactPolicy", policy =>
    {
        policy
            .WithOrigins("http://localhost:8081")
            .AllowAnyHeader()
            .AllowAnyMethod();
    });
});

var app = builder.Build();

// ... (Swagger, HTTPS redirection, etc. if present)

app.UseRouting();          // if explicitly present

app.UseCors("ReactPolicy");   // <-- MUST be here: after UseRouting, before
UseAuthorization/MapControllers

app.UseAuthorization();    // if present

app.MapControllers();

app.Run();
```

Key things to check:

1. Is app.UseCors(“ReactPolicy”); present at all?
2. Is it positioned after UseRouting() (if you have it) and before MapControllers()?
3. Is the policy name spelled identically in both places (“ReactPolicy”)?

Paste your actual Program.cs content (from var app = builder.Build(); to app.Run();) if you want me to confirm the exact order rather than guessing.

Then — Critical — Rebuild and Republish the Backend

CORS config changes won't take effect until you:

1. dotnet publish -c Release -o ./publish (or Visual Studio Publish)
2. Copy the new files to your backend's IIS folder (overwrite old ones)

3. In IIS Manager, restart the app pool: right-click Dapper_CRUD → Recycle (or Stop then Start)

Then hard refresh the frontend (Ctrl+Shift+R) and check the Network tab again for the Employee request — you should see Access-Control-Allow-Origin: http://localhost:8081 in the Response Headers, and the data should load.

B.20 Reviewing the Actual Program.cs

USER

```
using Dapper_CRUD.Data;
using Dapper_CRUD.Interface;
using Dapper_CRUD.Service;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

builder.Services.AddControllers();
// Learn more about configuring Swagger/OpenAPI at https://aka.ms/aspnetcore/swashbuckle
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();


builder.Services.AddScoped<DapperContext>();
builder.Services.AddScoped<IEmpService,EmpService>();


builder.Services.AddCors(options =>
{
   options.AddPolicy("ReactPolicy", policy =>
   {
      policy
         .WithOrigins("http://localhost:8081")
         .AllowAnyHeader()
         .AllowAnyMethod();
   });
});


var app = builder.Build();

// Configure the HTTP request pipeline.
```

> *if (app.Environment.IsDevelopment())*
> *{*
> *app.UseSwagger();*
> *app.UseSwaggerUI();*
> *}*
>
> *app.UseHttpsRedirection();*
>
> *app.UseCors("ReactPolicy");*
>
> *app.UseAuthorization();*
>
> *app.MapControllers();*
>
> *app.Run();*

Response

Your Program.cs structure is correct — order looks right. But there's one likely culprit: app.UseHttpsRedirection(); is placed before app.UseCors(...), and it runs unconditionally (not just in Development).

Here's the problem: your frontend is calling http://localhost:8080 (plain HTTP). If IIS/Kestrel is configured to redirect HTTP → HTTPS, the browser's CORS preflight/request gets redirected mid-flight, and the redirected response won't carry your CORS headers correctly — this exact symptom (200 OK but net::ERR_FAILED + no Access-Control-Allow-Origin) is classic for this.

Fix: Reorder or Remove HTTPS Redirection

Since you're serving over plain HTTP on port 8080 (no SSL binding set up), either remove this line entirely for now, or move CORS before it:

```
var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseCors("ReactPolicy");   // <-- moved above UseHttpsRedirection

app.UseHttpsRedirection();

app.UseAuthorization();
```

```
app.MapControllers();

app.Run();
```

Or simplest fix — just comment it out entirely since you're not using HTTPS on this IIS site right now:

```
// app.UseHttpsRedirection();
```

Then — This Is Critical — Did You Actually Republish?

You've now changed CORS config twice (added policy, then reordered). Each time, you must:

1. dotnet publish -c Release -o ./publish (or VS Publish)
2. Copy the fresh output into your backend's IIS folder, overwriting old files
3. In IIS Manager → Application Pools → right-click Dapper_CRUD → Recycle

If you only edited the code but never republished, the server is still running the old DLL with the old (or missing) CORS setup — that alone would explain everything you're seeing.

After republishing and recycling the pool, hard refresh the frontend (Ctrl+Shift+R) and check the Network tab again. Let me know what the Employee request shows this time.

B.21 Adding web.config for SPA Routing

> **USER**
> *Adding Web.Config*

Response

This web.config file needs to go directly inside your frontend's deployed folder — the same place your index.html and assets folder live:

```
C:\inetpub\wwwroot\dapper_frontend\web.config
```

So the folder should end up looking like:

```
C:\inetpub\wwwroot\dapper_frontend\
├── index.html
├── assets\
└── web.config   ← new file you're adding
```

How to Create It

1. Open Notepad (or VS Code, Notepad++, etc.)
2. Paste this content:

```
<configuration>
  <system.webServer>
    <rewrite>
      <rules>
        <rule name="React Routes" stopProcessing="true">
          <match url=".*" />
          <conditions logicalGrouping="MatchAll">
            <add input="{REQUEST_FILENAME}" matchType="IsFile" negate="true" />
            <add input="{REQUEST_FILENAME}" matchType="IsDirectory" negate="true" />
          </conditions>
          <action type="Rewrite" url="/index.html" />
        </rule>
      </rules>
    </rewrite>
  </system.webServer>
</configuration>
```

1. Save it as web.config — important: in the Save dialog, change “Save as type” to All Files, otherwise Notepad will save it as web.config.txt
2. Save it directly into C:\inetpub\wwwroot\dapper_frontend\

Before This Works, You Need URL Rewrite Module Installed

If you haven't already installed it: search “IIS URL Rewrite Module” on Microsoft's site, download and install the MSI on your server, then iisreset (or just restart the site) afterward.

Why You Need This

Right now, your homepage (/) loads fine. But if a user refreshes the page while on an inner route (like /add-employee or /employee/5), IIS will look for a physical file/folder matching that path, won't find one, and throw a 404 — because React Router handles those routes client-side, not IIS. This rule tells IIS “for any path that isn't a real file, just serve index.html and let React Router take over.”

Did your CORS fix work first, by the way — is the employee table showing data now?

B.22 Correct Location for web.config

> **USER**
> *into the new folder I created and deployed it on IIS or the frontend project folder?*

Response

Into the deployed IIS folder, not the project folder.

```
C:\inetpub\wwwroot\dapper_frontend\web.config
```

That's the folder you created and pointed the IIS site to — the one containing index.html and assets right now. This web.config is purely an IIS server-side config file; it has nothing to do with your React source

code, so it doesn't belong in your project folder (D:\Dapper\...\dapper_react\) at all — it only needs to exist wherever IIS is actually serving files from.

One thing to double check: since your dist build doesn't include a web.config by default, you'll need to re-add this file every time you rebuild and redeploy the frontend (since redeploying typically means copying fresh dist contents over the old ones, which could wipe it out if you delete-and-replace the whole folder). Keep a copy of this web.config saved somewhere handy so you can quickly drop it back in after each redeploy — or better, just copy it into your dist folder locally before copying to IIS, so it always travels along with your build output.

B.23 HTTP 500.19 — Configuration Data Invalid

> **USER**
> *After adding web.config in deployed frontend folder*
>
> *now got this issue:-*
>
> *HTTP Error 500.19 - Internal Server Error*
> *The requested page cannot be accessed because the related configuration data for the page is invalid.*
> *Detailed Error Information:*
> *Module IIS Web Core*
> *Notification Unknown*
> *Handler Not yet determined*
> *Error Code 0x8007000d*
> *Config Error*
> *Config File \\?\D:\IIS Frontend Publish\web.config*
> *Requested URL http://localhost:8081/*
> *Physical Path*
> *Logon Method Not yet determined*
> *Logon User Not yet determined*
> *Config Source:*
> *-1:*
> *0:*
> *More Information:*
> *This error occurs when there is a problem reading the configuration file for the Web server or Web application. In some cases, the event logs may contain more information about what caused this error.*
> *View more information »*

Response

This 500.19 error means IIS can't parse the web.config XML — usually one of these causes:

1. Most Likely: URL Rewrite Module Isn't Installed

If the URL Rewrite module isn't installed on this IIS server, IIS doesn't recognize the <rewrite> element at all and throws exactly this error (0x8007000d).

Check: Open IIS Manager → click on your frontend site → look in the middle pane for a URL Rewrite icon. If it's not there, the module isn't installed.

Fix: Download and install the IIS URL Rewrite Module from Microsoft:

```
https://www.iis.net/downloads/microsoft/url-rewrite
```

Install the MSI, then run iisreset (or just restart the site/app pool) afterward.

2. Check the File Wasn't Saved Incorrectly

Since you used Notepad, a common issue: the file got saved as web.config.txt instead of web.config, or the encoding introduced weird characters.

Check: In File Explorer, navigate to D:\IIS Frontend Publish\ (your actual physical path, based on the error), enable "Show file extensions" (View tab → check "File name extensions"), and confirm the file is literally named web.config — not web.config.txt.

Also open the file in Notepad again and confirm the content looks exactly like the XML with no extra characters, no duplicate <configuration> tags, nothing pasted twice.

3. Quickest Way to Isolate the Cause

Temporarily rename/move web.config out of that folder, confirm the site loads fine again (proving the file itself is the problem), then:

- Install URL Rewrite module first (likely fix)
- Put the file back
- Test again

Try installing the URL Rewrite module first — that's the most common cause of this specific error when a <rewrite> section is involved. Let me know if it's already installed or not.

B.24 Locating the URL Rewrite Icon

> **USER**
> *Where to click?*

Response

In IIS Manager:

1. In the left panel (Connections tree), expand down to Sites → click on your frontend site (e.g. dapper_frontend)

2. Once selected, look at the middle panel — it shows a bunch of icons grouped by category (ASP.NET, IIS, Management, etc.)
3. Look under the IIS section specifically for an icon labeled URL Rewrite

If you see it there → double-click it to open it, meaning the module is installed (the config error is likely something else — check the file name/encoding instead, as mentioned in point 2 of my last message).

If you don't see a “URL Rewrite” icon anywhere in that middle panel → the module is not installed, and that confirms the fix: download it from the link below and install it.

Direct Download Link

```
https://www.iis.net/downloads/microsoft/url-rewrite
```

Click that link → download the installer (choose the version matching your system, likely WebPI or the standalone x64 MSI) → run it → follow the install wizard → once done, restart IIS (iisreset in Command Prompt, or just restart the site).

Screenshot the middle panel of IIS Manager with your frontend site selected if you want me to confirm exactly what you're seeing.

B.25 Confirming the Steps to Check for the Module

> **USER**
> *In IIS Manager:*
>
> *In the left panel (Connections tree), expand down to Sites → click on your frontend site (e.g. dapper_frontend)*
> *Once selected, look at the middle panel — it shows a bunch of icons grouped by category (ASP.NET, IIS, Management, etc.)*
> *Look under the IIS section specifically for an icon labeled URL Rewrite*
> *If you see it there → double-click it to open it, meaning the module is installed (the config error is likely something else — check the file name/encoding instead, as mentioned in point 2 of my last message).*
>
> *If you don't see a "URL Rewrite" icon anywhere in that middle panel → the module is not installed, and that confirms the fix: download it from the link below and install it.*
> *Is this correct?*

Response

Confirmed — no “URL Rewrite” icon anywhere in that IIS section. Only Authentic..., Default Document, Directory Browsing, Error Pages, Handler Mappings, HTTP Respon..., MIME Types, Modules, Output Caching, Request Filtering, SSL Settings.

The URL Rewrite module is not installed. That's exactly what's causing your 500.19 error — IIS doesn't recognize the <rewrite> tag in your web.config at all.

Fix: Install the URL Rewrite Module

1. Go to this link in your browser:

```
https://www.iis.net/downloads/microsoft/url-rewrite
```

1. Click Install this extension (or the direct download button — it'll download a small MSI installer, e.g. rewrite_amd64_en-US.msi)
2. Run the downloaded MSI installer
3. Click through the install wizard (Next → Accept license → Install) — takes under a minute
4. Once done, restart IIS to make sure it picks up the new module:
    a. Open Command Prompt as Administrator
    b. Run:

```
iisreset
```

After Installing, Verify

Go back to IIS Manager → click on Dapper_Frontend site again → you should now see a URL Rewrite icon in the IIS section (middle panel).

Then refresh http://localhost:8081 in your browser — the 500.19 error should be gone and your site should load normally again.

Go ahead and install it, then let me know if the site loads.

B.26 Direct Download Link and Installation Steps

Response

Click the English x64 installer link (almost all modern Windows machines are 64-bit):

```
https://download.microsoft.com/download/1/2/8/128E2E22-C1B9-44A4-BE2A-
5859ED1D4592/rewrite_amd64_en-US.msi
```

Steps

1. Click that link — it'll download rewrite_amd64_en-US.msi
2. Once downloaded, open/run the MSI file
3. Click through the installer: Next → accept the license terms → Install
4. Click Finish when done
5. Open Command Prompt as Administrator (right-click → Run as administrator) and run:

```
iisreset
```

Then Verify

1. Go back to IIS Manager
2. Click on your Dapper_Frontend site
3. You should now see a URL Rewrite icon in the IIS section of the middle panel (it wasn't there before)

Then Test Your Site

Refresh http://localhost:8081 in your browser — the 500.19 error should be gone.

Note on x86 vs x64: if your Windows is 32-bit (rare nowadays), use the x86 link instead. You can check via Settings → System → About → look at "System type" — it'll say "64-bit operating system" or "32-bit operating system."

Go ahead and install it, then let me know what you see.

B.27 Clarifying Whether to Open the URL Rewrite Panel

> **USER**
> *Should i click on URL rewrite?*

Response

No need to click into it right now — just seeing the icon there confirms the module installed successfully.

You can leave it as is (the web.config rule you already added is already active automatically now that the module exists — you don't need to configure anything through this UI).

Just Go Test Your Site Now

1. Go to your browser
2. Navigate to http://localhost:8081
3. Hard refresh: Ctrl+Shift+R

The 500.19 error should be gone, and your Employee Management System should load normally. Try navigating to an inner route (if your app has one, like clicking into an employee or an add/edit page) and then refresh the browser on that page — that's the real test of whether the rewrite rule is working (confirms deep-link refreshes don't 404 anymore).

End of Session Log

This document is a verbatim, formatted record of the deployment guidance and troubleshooting exchanges covering: initial IIS setup for the backend and frontend, locating and copying the Vite build output, IIS site and application-pool configuration, the SPA URL-rewrite rule, resolving CORS blocking between the frontend (port 8081) and backend (port 8080), correcting a stale API base URL, and installing the IIS URL Rewrite module to resolve an HTTP 500.19 configuration error.